Re-measuring MMSpec: Results on Qwen2.5-VL-7B (600 questions)

Smit Thakare · Scaling LLMs final project · working note, November 10, 2026

Abstract. We re-ran the ten speculative decoding methods in the MMSpec benchmark (arXiv 2603.14989) on Qwen2.5-VL-7B over the full 600-question test split, under a single protocol: greedy decoding, throughput in tokens per second, paired bootstrap comparisons, and an explicit check that the output is unchanged. Acceptance lengths reproduce for eight of ten methods. Speed does not. Among methods with intact output, ViSpec and EAGLE-1 are indistinguishable at about 1.57×. MSD appears fastest only because its timer excludes the vision encoder; timed like the others it is significantly slower. SAM, Recycling and MSD corrupt 55%, 61% and 14% of answers respectively, and SAM's reported speedup disappears once output length is accounted for.

1 Setup

Speculative decoding lets a small drafter propose several tokens which the target model then verifies in a single forward pass. Under greedy decoding the accepted prefix is exactly what the target would have produced alone, so a correct method changes speed and nothing else. We use MMSpec's released code (commit 2904868) unmodified, writing any instrumentation as separate copies. All runs use one NVIDIA L4 and batch size 1 on the *test* split: 100 questions from each of six tasks drawn from GQA, TextVQA, COCO, CharXiv, MMMU-Pro, and ConvBench/MM-MT-Bench. MAT is the mean number of draft tokens accepted per verification step; it depends only on the models and so transfers across hardware. Speedup is pooled tokens per second relative to a pure greedy baseline. Intervals are 95% bootstrap ranges from 2,000 resamples of questions; comparisons between methods reuse the same resamples, so they test the difference directly.

Method	MAT ours / paper	Speedup ours (95% CI)	Speedup paper	Length vs. base	Corrupted answers	Junk after answer
ViSpec	2.27 / 1.29	1.58 (1.55–1.61)	1.51	1.01	0.3%	8.0%
EAGLE-1	2.19 / 2.36	1.56 (1.52–1.60)	2.11	0.99	0.5%	7.8%
EAGLE-2	1.70 / 1.78	1.54 (1.50–1.58)	2.02	1.00	0.0%	6.2%
MSD, own timer	2.67 / 2.57	1.66 (1.62–1.70)	2.58	0.87	14.0%	0.0%
MSD, full timer		1.47 (1.43–1.52)				
Medusa	0.77 / 0.80	1.09 (1.08–1.11)	1.49	1.00	0.3%	8.7%
PLD	0.16 / 0.17	1.00 (0.98–1.01)	1.05	1.02	0.2%	0.2%
Lookahead	0.37 / 0.33	0.97 (0.94–1.00)	1.07	1.00	0.0%	0.0%
SAM	0.26 / 0.23	0.97 (0.94–1.00)	2.17	0.48	55.3%	0.3%
Recycling	0.11 / 0.11	0.94 (0.93–0.95)	1.04	1.01	61.3%	0.0%
EAGLE-3	0.06 / 0.24	0.67 (0.67–0.68)	0.96	0.99	0.2%	0.0%

Table 1. All ten methods on 600 questions. Speedup is relative to pure greedy decoding (paper figures are on A100s, ours on an L4, so only MAT is directly comparable). *Full timer* adds the vision-encoder time that MSD's own timer leaves out. *Length* is total tokens generated relative to the baseline. *Corrupted*: doubled words, foreign-script fragments or stray tokens inside the answer, none of which the baseline produces. *Junk after answer*: a stray token appended after an otherwise complete answer.

2 Findings

Three methods are not lossless in the released code. SAM and Recycling corrupt more than half their answers with doubled words (“are are”), stray tokens such as *addCriterion*, or Chinese fragments in English text; MSD drops or garbles words in 14%, diverging from the baseline a median 4% of the way into the

answer. Every other method stays at or below 0.5%.

MSD's lead is an artifact of its timer. By its own timer MSD is the fastest method, ahead of EAGLE-1 by 0.10× (paired 95% CI +0.05 to +0.16). Its timer, however, starts after the vision encoder has run, work that every other method is charged for. Timed the same way, MSD falls to 1.47× and is significantly *slower* than EAGLE-1 (−0.08) and ViSpec (−0.11). Its per-task acceptance is also logged as a running average over the whole run, which hides real variation.

SAM's speedup comes from shorter output. SAM writes 0.48× as many tokens as the baseline, so it finishes 2.02× sooner by the clock while being no faster per token (0.97×). Its reported 6.53× on complex reasoning measures 1.03× here.

Among intact methods there is no clear winner. ViSpec and EAGLE-1 cannot be separated (difference −0.02, CI −0.06 to +0.02). ViSpec edges EAGLE-2 by a small but reliable 0.04. ViSpec accepts 2.27 tokens per step rather than the 1.29 MMSpec reports, in line with the 2.24 in the original ViSpec paper. EAGLE-3's near-zero acceptance coincides with a warning that its drafter config loads as the wrong model type.

The benchmark's baseline and batching differ from what they claim. MMSpec's baseline inherits Qwen's default repetition penalty of 1.05 while every speculative method decodes pure greedy; the two agree on only 8% of answers, so we compare against a pure greedy baseline. Its batch-size setting only reorders samples, so each request is still decoded alone.

Smaller issues in intact methods. EAGLE-1, EAGLE-2, ViSpec and Medusa append a stray token after an otherwise complete answer in 6–9% of cases. These four share a tree-verification path, and the pattern is consistent with the end-of-sequence token being accepted inside a draft and not cut cleanly; EAGLE-3, which almost never accepts a draft, never shows it. Separately, the intact methods reproduce the baseline's exact text on only 26–37% of questions, typically splitting about a fifth of the way into a fluent answer, which is most likely floating-point reordering flipping near-tied tokens.

3 Hypotheses and next steps

Drafters that see the image do not beat text-only ones on Qwen: ViSpec ties EAGLE-1, and MSD is slower once timed fairly. Copy-based methods do little for vision-language models, since an answer about an image rarely repeats text from the prompt (PLD 1.00×, Lookahead 0.97×). Whether speedups survive batched serving, and how they vary with image resolution, remain untested. Next we compare each method with its own non-speculative path to separate numeric drift from genuine errors, trace where the faulty methods accept wrong tokens, and carry the same rules into batched serving with vLLM.

Caveats. Our L4 is slower than the paper's A100s, so absolute speedups are not comparable; MAT and rankings are. All claims concern the released code, which may differ from what produced the published figures. The end-of-answer explanation is a hypothesis we have not yet confirmed in code.